2026-9-27 Linux网络--概念上

- 1. 为什么需要网络
  - 1.1 从独立计算机到网络互联
    - 1.1.1 独立模式
    - 1.1.2 网络互联
  - 1.2 局域网与广域网
    - 1.2.1 局域网 LAN
    - 1.2.2 广域网 WAN
    - 1.2.3 LAN 与 WAN 是相对概念
- 2. 网络通信为什么需要协议
  - 2.1 协议是通信双方共同遵守的约定
  - 2.2 仅有基本约定仍然不够
  - 2.3 为什么需要统一标准
- 3. 协议为什么需要分层
  - 3.1 复杂问题需要拆分
  - 3.2 分层的好处
  - 3.3 协议栈与对等通信
- 4. OSI 与 TCP/IP 网络模型
  - 4.1 OSI 七层模型
  - 4.2 TCP/IP 五层模型
    - 4.2.1 五层的基本职责
    - 4.2.2 从五层到四层
  - 4.3 设备通常工作在哪一层
- 5. 协议在软件中如何落地
  - 5.1 操作系统与 TCP/IP
  - 5.2 协议可以理解成双方认识的结构体
  - 5.3 报头与有效载荷
- 6. 局域网内如何完成一次通信
  - 6.1 MAC 地址标识局域网中的网卡
  - 6.2 从 ifconfig 观察主机网络
  - 6.3 以太网如何发送数据帧
  - 6.4 碰撞检测与碰撞避免
- 7. 数据如何逐层封装与交付
  - 7.1 发送：自上而下封装
  - 7.2 接收：自下而上解包与分用
  - 7.3 不同层次的数据名称

- 8. 小结
  - 8.1 网络概念的主线
  - 8.2 本笔记的边界

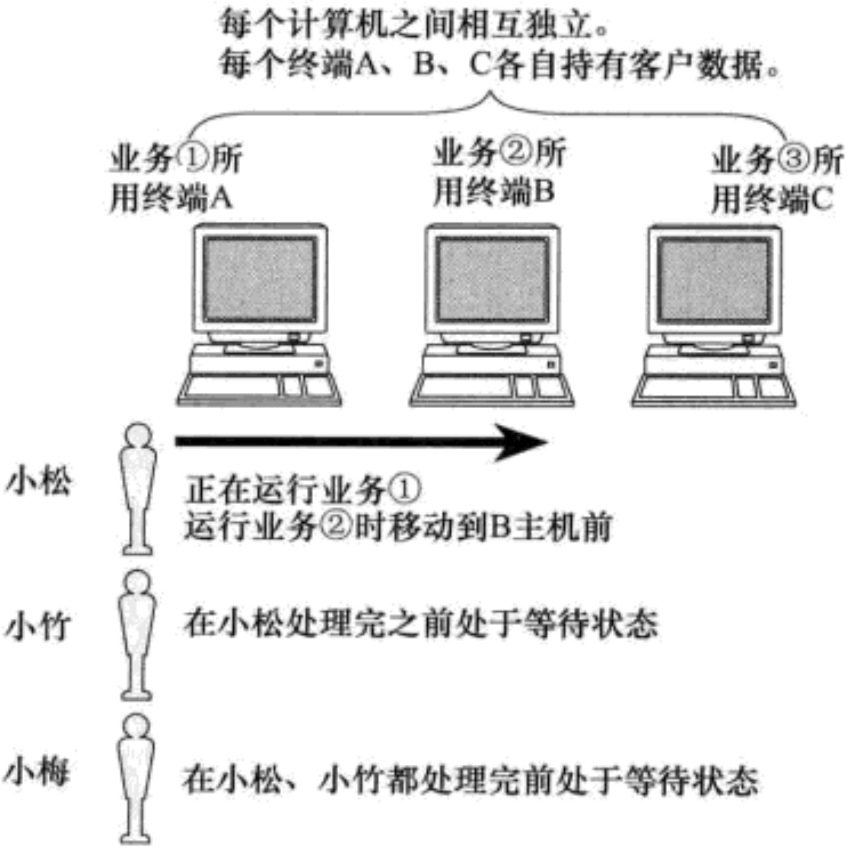
1. 为什么需要网络

网络不是凭空出现的软件功能。理解网络，可以先沿着“计算机为什么需要连接、连接之后还要解决什么问题”的路径展开。

1.1 从独立计算机到网络互联

1.1.1 独立模式

在独立模式下，每台计算机只处理自己的数据，彼此之间没有数据通路。终端 A、B、C 各自保存自己的数据，用户更换终端或业务时，数据也无法自然共享。

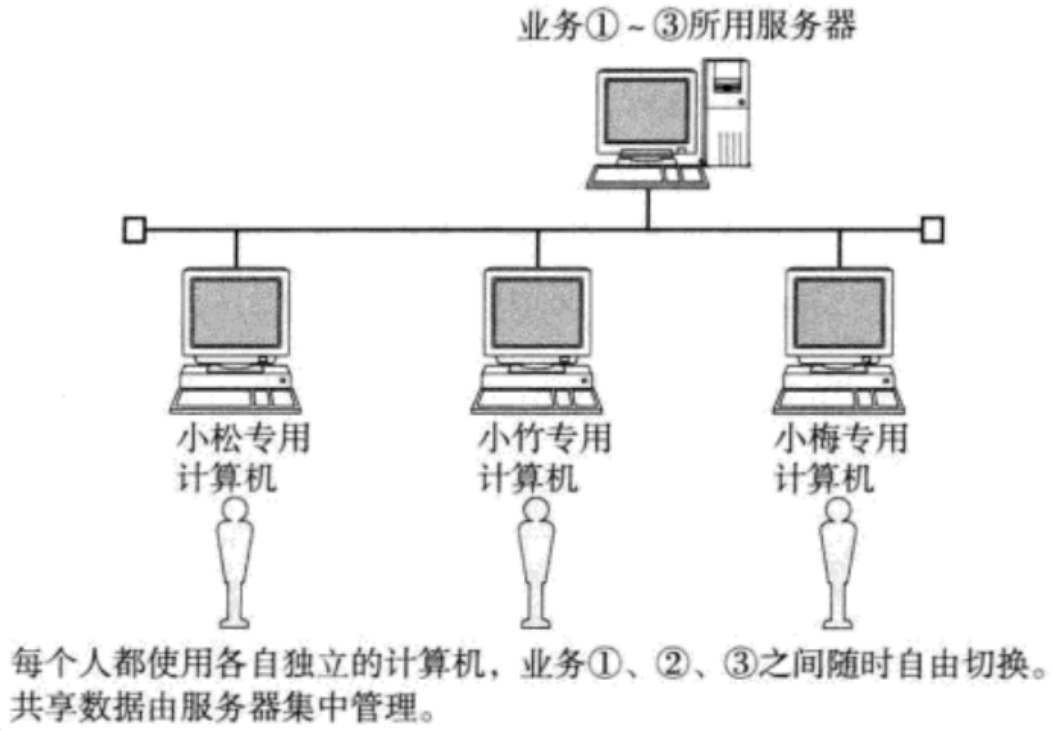


独立模式并不代表计算机不能工作，它的限制在于：

- 数据被限制在单台计算机内部；
- 多个人、多台设备之间无法自动协作；
- 一旦需要共享数据，只能依赖人工搬运或其他离线方式。

1.1.2 网络互联

把多台计算机连接起来后，不同终端可以访问共享资源，也可以互相传递数据。网络把“单独完成业务”变成了“多台主机协同完成业务”。

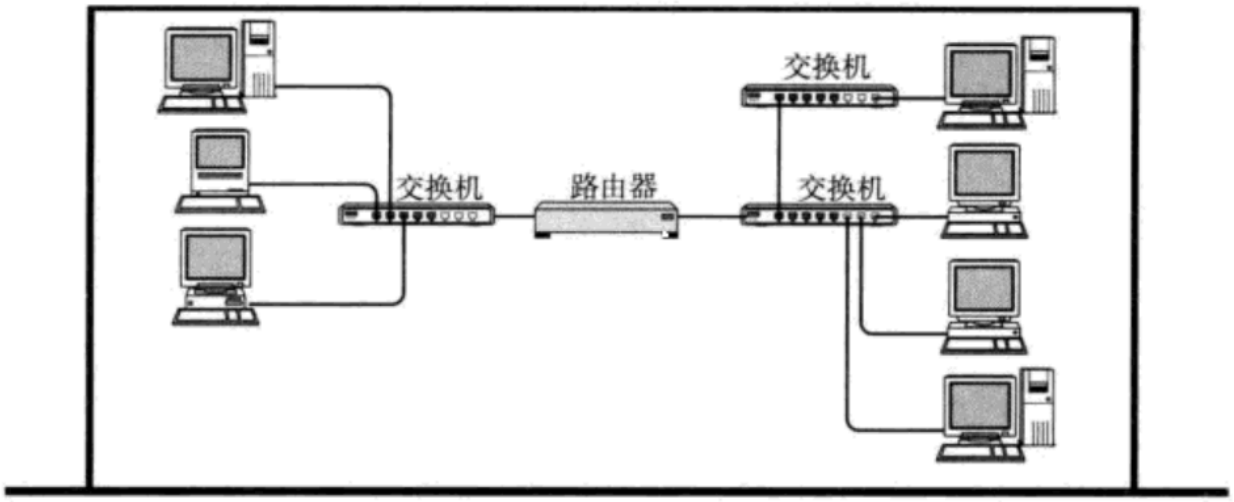


早期可以通过一台服务器集中保存业务数据，多个用户分别使用自己的计算机访问服务器。随着计算机数量增加，仅靠少量设备直接连接已经不够，交换机和路由器开始承担局域网内部的连接与不同网络之间的转发。

1.2 局域网与广域网

1.2.1 局域网 LAN

局域网（Local Area Network，LAN）覆盖教室、办公室、公司等较小范围。常见结构是计算机先接入交换机，再由路由器连接其他网络。



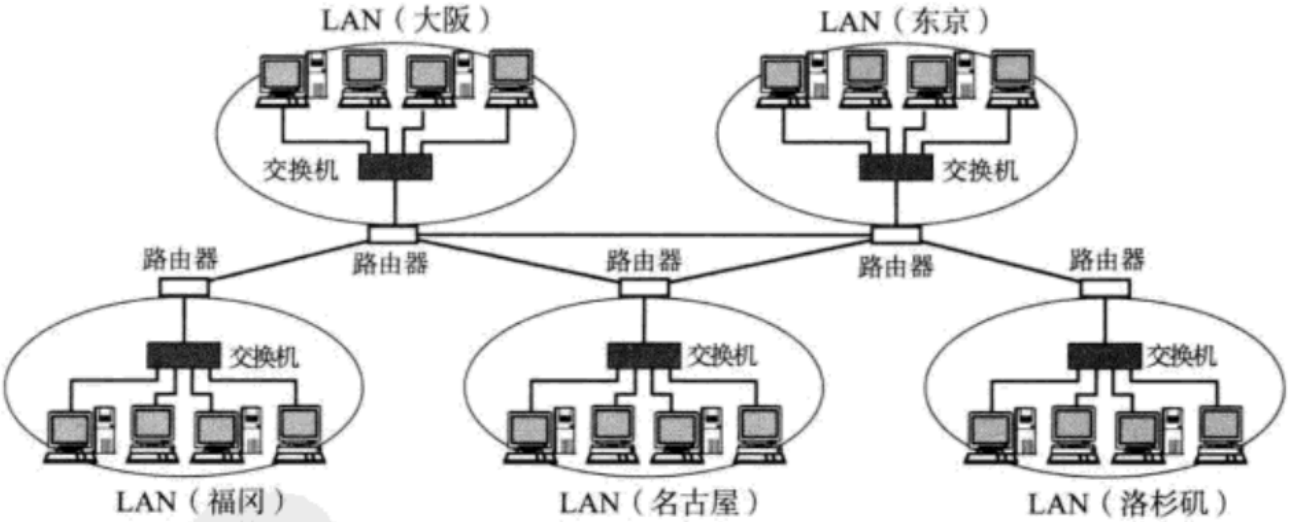
一栋楼或大学校园中有限的、狭小的、区域内网络。

LAN 的主要特点：

- 覆盖范围较小；
- 主机之间距离近，通常由统一组织管理；
- 交换机负责局域网内部的数据帧转发；
- 路由器负责把局域网连接到其他网络。

1.2.2 广域网 WAN

广域网（Wide Area Network，WAN）把远隔千里的主机或局域网连接起来，覆盖城市、国家甚至全球范围。



跨接相距较远的计算机或LAN的网络。

WAN 可以理解为“多个 LAN 再通过网络设备连接起来”。跨城市、跨国家的通信通常要经过多台路由器，而不能只依靠本地交换机完成。

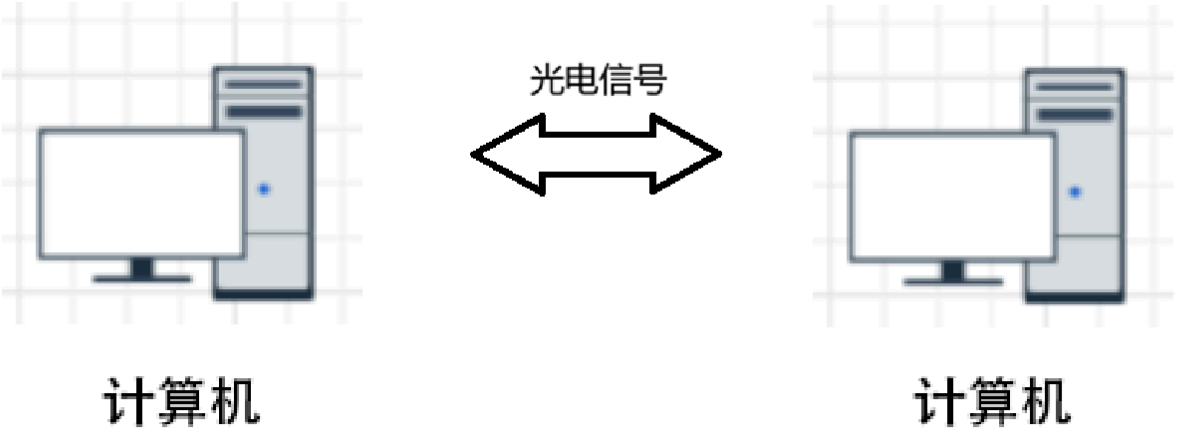
1.2.3 LAN 与 WAN 是相对概念

LAN 和 WAN 没有一条绝对不变的边界。一个学校网络相对于机房是广域网，一个国家级网络相对于跨国网络又可能被视为一个较大的局域网。因此，更重要的是判断网络的覆盖范围、管理边界和转发路径，而不是死记范围数字。

2. 网络通信为什么需要协议

2.1 协议是通信双方共同遵守的约定

“能连上”只说明双方之间存在物理通路，并不代表双方能够理解彼此发送的数据。



计算机之间通过光信号、电信号或电磁波传递信息，底层最终都表现为 `0` 和 `1`。但同样一位 `0` 或 `1`，可能由不同频率、不同电平、不同持续时间和不同组合方式表示。如果双方没有共同约定，收到的只是一串无法解释的波形。

协议需要规定的不只是“发送了哪些比特”，还包括：

- 数据由哪些字段组成；
- 每个字段占多少字节、采用什么顺序；
- 每个字段表示什么含义；
- 何时开始发送、何时结束；
- 收到数据后是否需要回复；
- 出错时如何处理。

可以用打电话前约定铃响次数来类比。生活里的约定是“响几声代表什么”，计算机网络里的协议则是双方都遵循的数据格式和交互规则。

2.2 仅有基本约定仍然不够

假设双方都知道“约好协议”，但发送方用频率表示 `01`，接收方却用电压强弱表示 `01`，双方仍然无法正常通信。这和两个人虽然都愿意交流、却分别使用不同语言一样。

因此，一个可用的协议不仅需要“双方愿意遵守”，还需要对编码方式、字段结构、处理顺序等细节作出一致规定。协议越完整，通信双方对同一串数据的理解越不容易产生歧义。

2.3 为什么需要统一标准

计算机厂商、操作系统和网络硬件种类都很多。如果每台设备只支持自己定义的格式，不同厂商的设备就无法协作。统一标准的价值，是让来自不同厂商、运行不同操作系统的设备，只要遵循同一套协议，就能完成通信。

这里要区分两个层面：

- **操作系统可以不同**：Windows、Linux 等系统内部的实现方式可以不同；
- **协议语义必须一致**：双方对字段、状态和收发顺序的理解必须相同。

这也是网络协议能够跨越不同主机、不同系统甚至不同硬件的基础。

3. 协议为什么需要分层

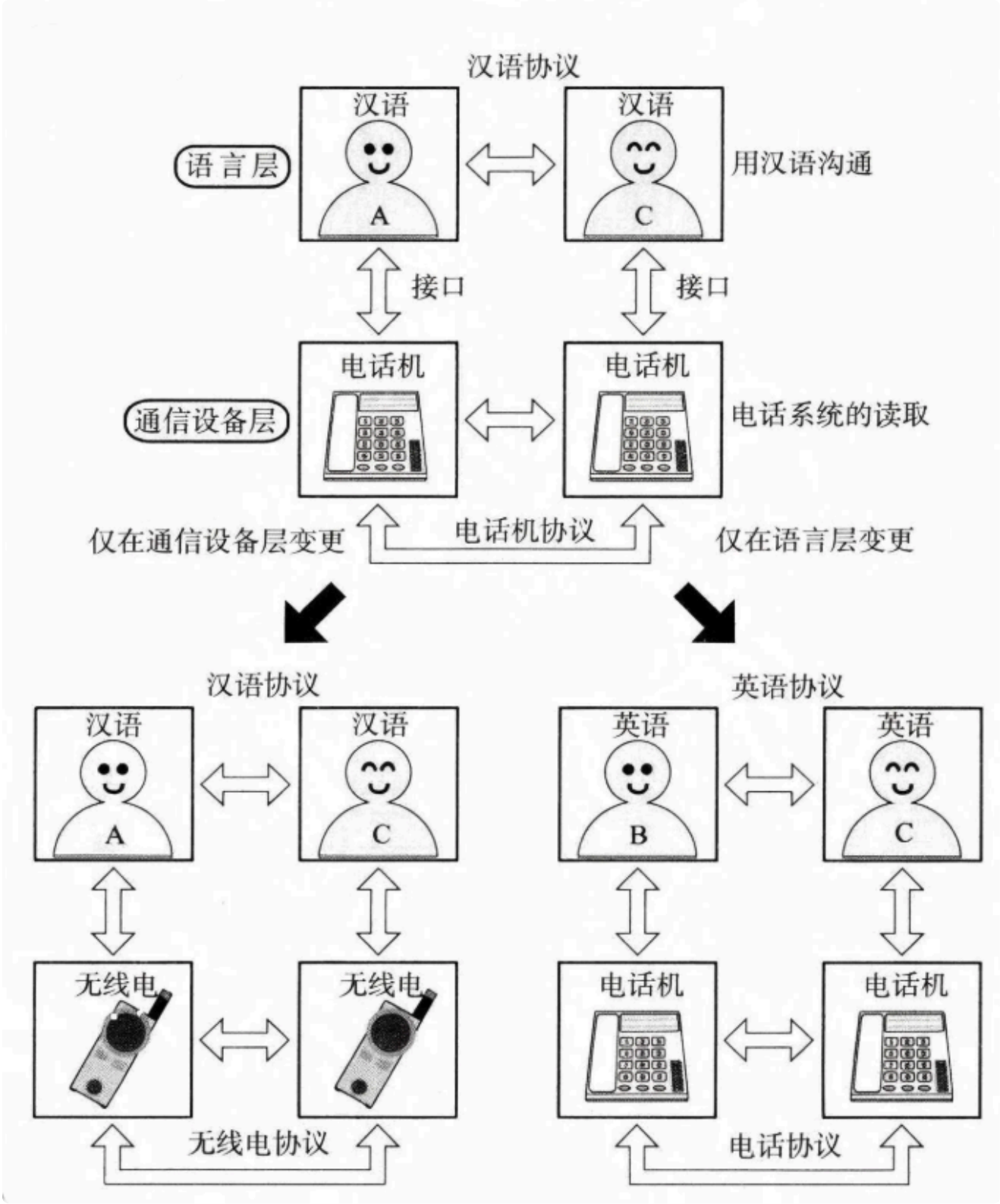
3.1 复杂问题需要拆分

网络通信同时涉及远端传输、路径选择、数据丢失、不同硬件介质、不同应用程序等问题。把所有规则写成一个不可拆分的整体，会导致任何一个局部变化都影响整个系统。

参考图给出的思路是：**网络通信这个大问题可以拆成多个性质不同的小问题，再为每类问题设置一层协议。**

例如，“使用什么语言表达内容”和“通过电话机、无线电还是光纤传递信号”是两个不同问题。可以分别定义语言层和通信设备层：语言层负责双方能理解的表达方式，通信设备层负责把内容编码成可传输的信号。

3.2 分层的好处



分层带来的主要收益是：

- **解耦合**：上层不需要关心下层如何传输，下层也不需要理解上层业务；
- **模块化**：每层只处理职责范围内的问题；
- **可替换**：更换某一层的实现时，其他层尽可能不受影响；
- **易维护**：出现问题时可以按层定位，而不是在一个庞大系统中查找。

“增加一层抽象”是一种常见的软件设计方法。参考图中的快递流程也可以帮助理解：揽收、分拣、运输、派送各自负责一段任务，最终共同完成整个交付过程。

3.3 协议栈与对等通信

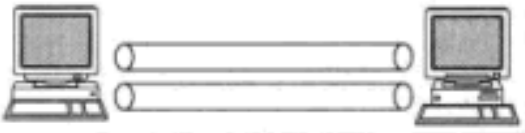
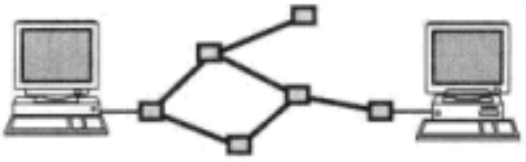
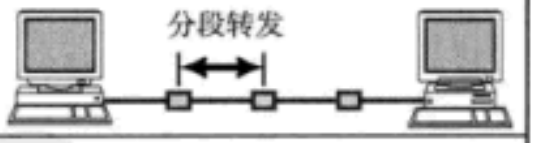
多台主机使用同一套分层协议时，每一层都通过与对方主机的对应层“对话”来完成任务。例如应用层使用应用协议、传输层使用传输协议、网络层使用 IP 协议。

这种逐层组合起来的协议体系称为**协议栈**。发送方自上而下交付，接收方自下而上交付；真正横向传输的是数据，协议语义则是在同层之间对应。

4. OSI 与 TCP/IP 网络模型

4.1 OSI 七层模型

OSI（Open System Interconnection，开放系统互连）七层模型是一种逻辑上的网络参考模型。它把网络通信拆为七层，用于说明不同层次应承担的职责。

	分层名称	功　能	每层功能概览
7	应用层	针对特定应用的协议。	针对每个应用的协议 电子邮件 ↔ 电子邮件协议 远程登录 ↔ 远程登录协议 文件传输 ↔ 文件传输协议
6	表示层	设备固有数据格式和网络标准数据格式的转换。	 接收不同表现形式的信息，如文字流、图像、声音等
5	会话层	通信管理。负责建立和断开通信连接（数据流动的逻辑通路）。 管理传输层以下的分层。	何时建立连接，何时断开连接以及保持多久的连接？ 
4	传输层	管理两个节点之间的数据传输。负责可靠传输（确保数据被可靠地传送到目标地址）。	是否有数据丢失？ 
3	网络层	地址管理与路由选择。	经过哪个路由传递到目标地址？ 
2	数据链路层	互连设备之间传送和识别数据帧。	 数据帧与比特流之间的转换 
1	物理层	以“0”、“1”代表电压的高低、灯光的闪灭。 界定连接器和网线的规格。	 比特流与电子信号之间的切换  连接器与网线的规格

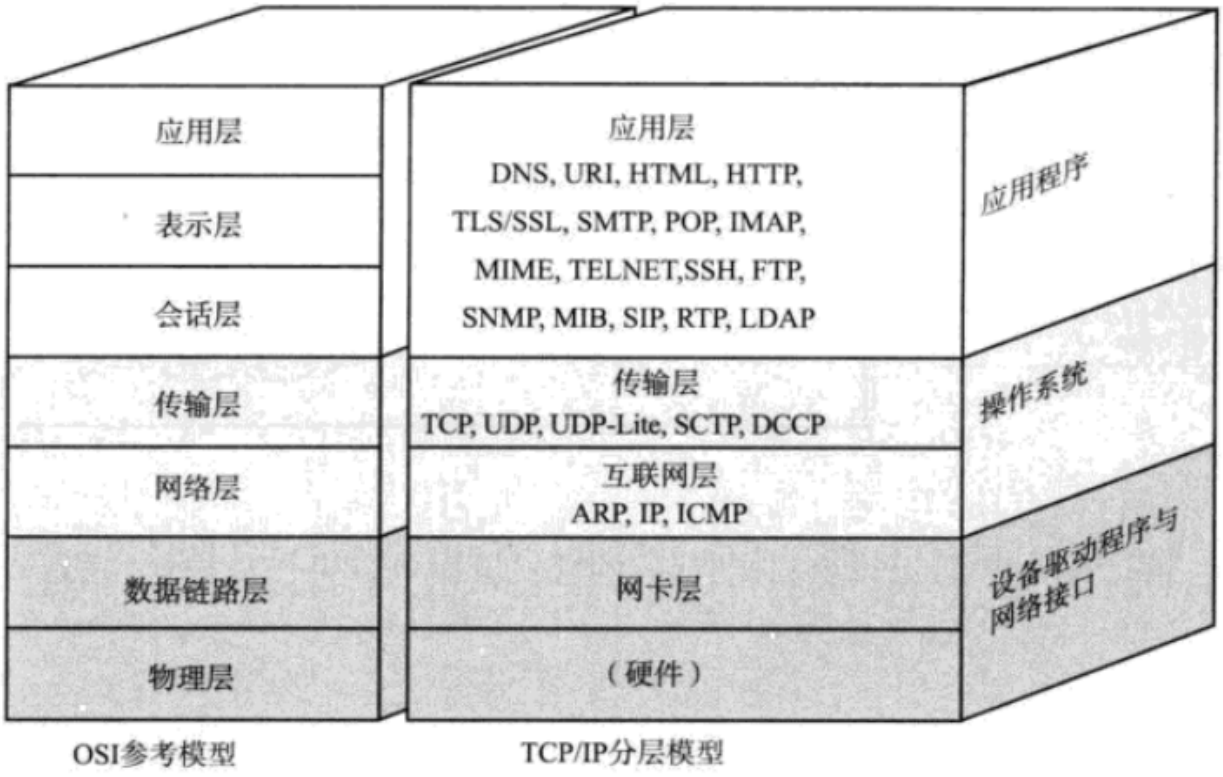
OSI 的价值主要在于：

- 用统一框架描述网络通信；
- 把服务、接口和协议区分开来；
- 帮助不同系统理解彼此的网络职责；
- 为后续学习具体协议提供分层地图。

OSI 模型更偏理论。工程实现中，通常会把会话层、表示层和应用层合并到同一上层区域，因此实际落地时常使用 TCP/IP 五层或四层模型。

4.2 TCP/IP 五层模型

TCP/IP 不是单个协议，而是一组协议的统称，也是构成互联网通信的主要协议体系。



4.2.1 五层的基本职责

层级	主要职责	典型协议或设备
应用层	为具体应用提供通信能力，如网页、邮件、文件传输、域名解析	HTTP、DNS、SMTP、FTP
传输层	在通信双方之间传输数据，处理可靠性、端到端交付等问题	TCP、UDP
网络层	为主机编址、选择跨网络路径并转发数据	IP、ICMP；路由器
数据链路层	在相邻设备之间传输和识别数据帧，处理局域网寻址与介质访问	Ethernet、Wi-Fi；交换机、网卡

层级	主要职责	典型协议或设备
物理层	把比特转换为电信号、光信号或电磁波，并规定接口和介质	双绞线、光纤、无线电；集线器

4.2.2 从五层到四层

物理层更偏硬件和通信介质，网络编程时关注得相对较少，因此常把数据链路层与物理层统称为“网络接口层”，形成 TCP/IP 四层模型：

1. 应用层；
2. 传输层；
3. 网络层；
4. 网络接口层。

五层与四层不是两套互不相容的知识，只是对底层粒度的不同划分。

4.3 设备通常工作在哪一层

从主要职责看：

- 集线器主要工作在物理层；
- 交换机主要工作在数据链路层；
- 路由器主要工作在网络层；
- 主机通常实现从应用层到物理层的完整协议栈。

这种划分描述的是设备的“主要工作位置”，并不是绝对限制。现代交换机和路由器可能同时实现多层功能，例如三层交换、端口转发等。分层分析时应关注设备实际处理的数据和字段，而不是只凭设备名称下结论。

5. 协议在软件中如何落地

5.1 操作系统与 TCP/IP

网络协议是软件，但并不是所有协议都运行在用户程序中。发送方最终必须把数据交给操作系统；操作系统再将数据交给网络驱动和网卡。

不同主机可以运行不同操作系统，但只要它们实现相同的协议语义，就可以互相通信。TCP/IP 的核心部分通常由操作系统内核、网络驱动和网络接口共同实现，应用程序则主要处理应用层协议和业务数据。

这也解释了一个常见现象：浏览器、聊天软件等用户程序无需自己驱动网卡，只需通过操作系统提供的网络能力发送和接收数据。

5.2 协议可以理解成双方认识的结构体

从代码角度看，一层协议可以被理解为一组双方共同认识的数据结构。例如：

```
struct protocol {
    int a;
    int b;
    int c;
};
```

如果发送方和接收方都按照相同的字段顺序、字段大小和含义解释这个结构体，那么同一段字节流就能被双方解析为相同的 `a`、`b`、`c`。

具体协议通常不会只有三个字段，但核心思想相同：**协议 = 双方共同遵守的数据格式 + 交互规则。**

5.3 报头与有效载荷

真实数据在协议栈中传输时，并不只是应用原始内容。每一层都会在上一层交付的内容前增加自己的控制信息，这部分信息称为**报头**（header）；被承载的原始内容称为**有效载荷**（payload）。

可以把它类比为快递单：

- 快递单记录收件人、发件人、物品信息；
- 包裹内部才是真正要交付的物品；
- 每一层只读取自己负责的标签，再把剩余内容交给下一层。

因此可以简写为：

```
报文 = 报头 + 有效载荷
```

报头里通常包含源地址、目的地址、数据长度、上层协议类型、校验信息等字段，具体内容由对应协议决定。

6. 局域网内如何完成一次通信

6.1 MAC 地址标识局域网中的网卡

数据链路层需要知道“这一帧应该交给哪个相邻设备”，局域网中通常使用 MAC 地址完成标识。

- MAC 地址长度为 48 位，即 6 字节；
- 通常写成 6 组十六进制数，例如 `00:16:3e:10:cc:36`；
- 正常网卡通常出厂时已分配 MAC 地址，常见情况下一个网卡对应一个 MAC 地址；
- 软件可以覆盖或伪装 MAC 地址，因此不能把“出厂固定”理解为绝对无法改变。

在经典共享式以太网中，多台主机连接在同一物理介质上。发送方把源 MAC 和目的 MAC 写入数据帧，其他主机也会收到信号，但只有目的 MAC 与自身匹配的网卡才会继续向上交付。

6.2 从 `ifconfig` 观察主机网络

参考图中的 `ifconfig` 输出同时展示了不同层次的信息：

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.28.71.17  netmask 255.255.240.0  broadcast 172.28.79.255
        ether 00:16:3e:10:cc:36  txqueuelen 1000  (Ethernet)
```

这些字段可以分成两类：

- `inet`、`netmask`、`broadcast`：与网络层地址和当前网段有关；
- `ether`：网卡的 MAC 地址，属于数据链路层；
- `UP`、`RUNNING`、`RX`、`TX`：接口状态以及接收、发送统计。

现代 Linux 也常使用 `ip addr` 和 `ip link show` 查看相同类型的信息。不同命令的展示格式会变化，但“IP 地址用于跨网络定位，MAC 地址用于局域网链路交付”的分工不变。

6.3 以太网如何发送数据帧

同一个共享式以太网可以看成一个广播介质。主机 A 想向主机 E 发送数据时，可以先把一帧发送到网络上：

```
源 MAC | 目的 MAC | 数据 | ...
```

发送后大致发生以下过程：

1. 局域网内其他主机都能收到这个帧；
2. 各网卡检查目的 MAC 是否与自身匹配；
3. MAC 不匹配的主机丢弃该帧；
4. 目标主机接收并交给上层协议，形成回应。

如果网络中使用了交换机，交换机会根据 MAC 地址表把已知单播帧转发到对应端口，而不必像共享式集线器那样把每一帧扩散到所有端口。但“根据目标 MAC 判断交给谁”的基本思想仍然保留。

6.4 碰撞检测与碰撞避免

在早期的共享式以太网、半双工链路中，同一时刻只能有一个主机成功占用介质。如果多台主机同时发送，电信号会叠加并造成**数据碰撞**。

为了共享同一介质，发送方需要：

- 检测介质是否空闲；
- 在发送过程中检测是否发生碰撞；
- 检测到碰撞后停止发送，等待一段随机时间再重试；
- 通过退避机制减少多台主机持续同时重试的概率。

可以把这一过程理解为“多人共用一条线路时，必须维护某一时刻只有一个发送者的原子性”。发生碰撞时通信不会立即成功，而是在检测、退避和重试中逐步完成任务。

现代交换式以太网常使用全双工链路，每个端口可以同时收发，碰撞检测的重要性已经下降。CSMA/CD 仍然适合用来理解共享介质的通信思想，但不能不加条件地说“今天的以太网每次发送都要检测碰撞”。

7. 数据如何逐层封装与交付

7.1 发送：自上而下封装

发送方的应用产生原始数据，例如文本“你好”。数据沿协议栈向下传递时，每一层都会加入当前层的报头：

```
应用数据
应用层报头 | 应用数据
传输层报头 | 应用层报头 | 应用数据
网络层报头 | 传输层报头 | 应用层报头 | 应用数据
链路层报头 | 网络层报头 | 传输层报头 | 应用层报头 | 应用数据
```

这个过程称为**封装**（Encapsulation）。最终形成的帧通过网卡发送到物理介质上。

7.2 接收：自下而上解包与分用

接收方从网卡得到帧后，按相反方向处理：

1. 数据链路层检查并去掉链路层报头；
2. 根据报头中的类型，把有效载荷交给网络层；
3. 网络层去掉自己的报头后，交给对应传输层协议；
4. 传输层去掉报头后，根据标识把数据交给对应应用；
5. 应用最终得到原始数据。

去掉报头并交给正确上层协议的过程称为**解包**；根据报头中的类型或标识选择一个上层处理者的过程称为**分用**。

因此，每一次网络通信都可以概括为：

```
发送端：逐层加报头 -> 网卡发送
接收端：网卡接收 -> 逐层去报头 -> 交给正确进程
```

7.3 不同层次的数据名称

同一条数据在协议栈的不同位置有不同名称：

所在位置	常见名称
应用层	数据、报文
传输层	段（segment）或数据报（datagram）
网络层	数据报或分组（packet）
数据链路层	帧（frame）
物理层	比特流（bit stream）

不同教材的命名会有差异，但应抓住共同结构：每一层都包含“本层控制信息 + 上层交付的数据”。

8. 小结

8.1 网络概念的主线

可以用一条主线串联本笔记：

1. 计算机因共享数据和协作而产生联网需求；
2. 多台主机通信需要共同遵守协议；
3. 复杂通信问题通过分层被拆成多个较小问题；
4. OSI 提供理论分层框架，TCP/IP 是实际广泛使用的协议体系；
5. 协议在软件中表现为双方共同认识的数据结构和交互规则；
6. 局域网中依靠 MAC 地址完成链路层交付，并处理介质共享与碰撞；
7. 数据发送时逐层封装，接收时逐层解包并分用。

8.2 本笔记的边界

参考图在“局域网通信”和“数据封装、解封装”之后，把下一步明确划分为“跨网络通信”。因此，本笔记停在：

- 数据如何进入本局域网；
- 接收方如何根据 MAC 地址接收并向上交付；
- 每一层如何通过报头和有效载荷完成封装与解封装。

IP 地址如何支持跨网段路由、端口如何区分进程，以及 Socket 如何编程，属于后续“跨网络通信”和网络编程部分，应在概念下篇或 Socket 专题中继续展开。